LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 6 – INHERITANCE



NAMA : Ruhfaidah Shirotul Akhiroh

NIM : 103132400019

KELAS : S1SD-205-REG-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

a. Inheritance

Inheritance merupakan suatu cara untuk menurunkan suatu kelas yang lebih umum menjadi suatu kelas yang lebih spesifik. Inheritance adalah salah satu ciri utama suatu bahasa program yang berorientasi pada objek, dan Java pasti menggunakannya. Java mendukung inheritance dengan memungkinkan satu kelas untuk bersatu dengan kelas lainnya ke dalam deklarasinya. Dalam inheritance terdapat dua istilah yang sering digunakan. Kelas yang menurunkan disebut kelas dasar (base class/super class), sedangkan kelas yang diturunkan disebut kelas turunan (derived class/subclass). Karakteristik pada superclass akan dimiliki juga oleh subclass-nya.

b. Keyword Super

Keyword super digunakan untuk merefer superclass dari suatu kelas, yaitu untuk merefer member dari suatu superclass, baik atribut maupun method.

c. Overloading Method

Di dalam java Anda diperbolehkan membuat dua atau lebih method dengan nama yang sama dalam satu kelas, tetapi jumlah dan tipe data argumen masing – masing method haruslah berbeda satu dengan yang lainnya. Hal itu yang dinamakan dengan Overloading Method.

d. Overiding Method

Di dalam java, jika dalam suatu subclass Anda mendefinisikan sebuah method yang sama dengan yang dimiliki oleh superclass, maka method yang Anda buat dalam subclass tersebut dikatakan meng-override superclass-nya. Sehingga jika Anda mencoba memanggil method tersebut dari instance subclass yang Anda buat, maka method milik subclass-lah yang akan dipanggil, bukan lagi method milik superclass-nya.

B. Guided

1. inheritance

```
package com.week6.Guided.gd1;

public class A {
    int x;
    int y;

    void tampilXY(){
        System.out.println("Nilai x = "+ x +", y = "+ y);
    }
}
```

```
package com.week6.Guided.gd1;

public class B extends A {  //Child dari class A
    int z;
    int hasil;

    void jumlahXYZ(){
        hasil = (x+y+z);

        System.out.println("Jumlah (x+y+z) = " + hasil);
    }
}
```

```
package com.week6.Guided.gd1;

public class Demo {
    public static void main(String[] args){
        A superclass = new A();
        B subclass = new B();

        superclass.x = 5;
        superclass.y = 3;

        System.out.println("Superclass: ");
        superclass.tampilXY();

        subclass.x = 1;
        subclass.y = 2;
        subclass.z = 5;

        System.out.println("Subclass: ");
        subclass.jumlahXYZ();
        subclass.tampilXY();
    }
}
```

Output :

```
Superclass:
Nilai x = 5, y = 3
Subclass:
Jumlah (x+y+z) = 8
Nilai x = 1, y = 2
------------------------------------------
BUILD SUCCESS
------------------------------------------
Total time:  1.307 s
Finished at: 2026-10-23T15:11:48+07:00
------------------------------------------
```

2. Keyword Super

```
package com.week6.Guided.gd2;

public class Employee {
    private String name;
    String departemen;

    public Employee(String name){
        this.name = name;
    }

    public void tampilNama(){
        System.out.println("nama: "+ name);
    }
}
```

```
package com.week6.Guided.gd2;

public class Manager extends Employee {
    private String alamat;

    public Manager(String nama, String alamat){
        super(nama);
        this.alamat = alamat;
    }

    public void tampilInformasi(){
        super.tampilNama();
        System.out.println("alamat: "+ alamat);
        System.out.println("departemen: "+ super.departemen);
    }
}
```

```
package com.week6.Guided.gd2;

public class Demo {
    public static void main(String[] args){
        Manager adi = new Manager("Adi", "Sukabirus");
        adi.departemen = "SDM";
        adi.tampilInformasi();
    }
}
```

Output :

```
nama: Adi
alamat: Sukabirus
departemen: SDM
------------------------------------------
BUILD SUCCESS
------------------------------------------
Total time:  1.085 s
Finished at: 2026-10-23T15:14:21+07:00
------------------------------------------
```

3. Overloading Method

```
package com.week6.Guided.gd3;

public class Lagu {
    String pencipta;
    String judul;

    void isiParam(String param1){
        judul = param1;
        pencipta = "Tidak Dikenal";
    }

    void isiParam(String param1, String param2){
        judul = param1;
        pencipta = param2;
    }

    void tampilkan(){
        System.out.println("Judul: "+ judul +", Pencipta; "+ pencipta);
    }
}
```

```
package com.week6.Guided.gd3;

public class Main {
    public static void main(String[] args) {
        Lagu d,e;
        d = new Lagu();
        e = new Lagu();

        d.isiParam("Lagu 1");
        e.isiParam("Lagu 2", "aku");

        d.tampilkan();
        e.tampilkan();
    }

}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ modul_06 ---
Judul: Lagu 1, Pencipta; Tidak Dikenal
Judul: Lagu 2, Pencipta; aku
------------------------------------------------
BUILD SUCCESS
------------------------------------------------
Total time:  1.400 s
Finished at: 2026-10-23T15:17:51+07:00
------------------------------------------------
```

4. Overiding Method

```
package com.week6.Guided.gd4;

public class A {
    public void tampilkan(){
        System.out.println("Method milik class A dipanggil....");
    }
}
```

```
package com.week6.Guided.gd4;

public class B extends A {
    public void tampilkan(){
        System.out.println("Method milik class B dipanggil....");
    }
}
```

```
package com.week6.Guided.gd4;

public class Main {
    public static void main(String[] args){
        B overidB = new B();
        overidB.tampilkan();
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ modul_06 --
Method milik class B dipanggil....
------------------------------------------------
BUILD SUCCESS
------------------------------------------------
Total time:  1.469 s
Finished at: 2026-10-23T15:20:13+07:00
------------------------------------------------
```

C. Unguided

1. Soal Unguided 1

   Sebuah tim machine learning ingin membuat program untuk membandingkan estimasi biaya komputasi dari beberapa jenis model yang akan dilatih. Ada tiga jenis model: model regresi, model klasifikasi, dan model klasifikasi berbasis deep learning yang merupakan pengembangan dari model klasifikasi biasa.

   Buatlah program Java dengan ketentuan: Penamaan kelas, atribut, dan method ditentukan sendiri oleh Anda dan semua atribut bersifat private.

   Jawaban :

```
package com.week6.Unguided;

public class ModelUmum {
    private String nama;
    private int jumlahDataLatih;

    public ModelUmum(String nama, int jumlahDataLatih) {
        this.nama = nama;
        this.jumlahDataLatih = jumlahDataLatih;
    }

    public int getJumlahDataLatih() {
        return jumlahDataLatih;
    }

    public void tampilInfo() {
        System.out.println("Model: " + nama + " | Data latih: " + jumlahDataLatih);
    }

    // Overloading 1: tanpa parameter
    public int hitungBiaya() {
        return jumlahDataLatih * 2;
    }

    // Overloading 2: dengan parameter epoch
    public int hitungBiaya(int epoch) {
        return jumlahDataLatih * epoch;
    }
}
```

```
package com.week6.Unguided;

public class ModelRegresi extends ModelUmum {
    private int jumlahFitur;

    public ModelRegresi(String nama, int jumlahDataLatih, int jumlahFitur) {
```

```
        super(nama, jumlahDataLatih);
        this.jumlahFitur = jumlahFitur;
    }

    @Override
    public void tampilInfo() {
        super.tampilInfo();
        System.out.println("Jumlah fitur: " + jumlahFitur);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahFitur * 10);
    }

    // Overloading 3: epoch dan ukuran batch
    public int hitungBiaya(int epoch, int ukuranBatch) {
        return getJumlahDataLatih() * epoch / ukuranBatch;
    }
}
```

```
package com.week6.Unguided;

public class ModelKlasifikasi extends ModelUmum {
    private int jumlahKelas;

    public ModelKlasifikasi(String nama, int jumlahDataLatih, int jumlahKelas) {
        super(nama, jumlahDataLatih);
        this.jumlahKelas = jumlahKelas;
    }

    @Override
    public void tampilInfo() {
        super.tampilInfo();
        System.out.println("Jumlah kelas: " + jumlahKelas);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() * jumlahKelas;
    }
    // hitungBiaya(int epoch) tidak di-override
}
```

```
package com.week6.Unguided;

public class ModelDeepLearning extends ModelKlasifikasi{
    private int jumlahLayer;
```

```
    public ModelDeepLearning(String nama, int jumlahDataLatih, int jumlahKelas, int
jumlahLayer) {
        super(nama, jumlahDataLatih, jumlahKelas);
        this.jumlahLayer = jumlahLayer;
    }

    @Override
    public void tampilInfo() {
        super.tampilInfo();
        System.out.println("Jumlah layer: " + jumlahLayer);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahLayer * 200);
    }
}
```

```
package com.week6.Unguided;

public class Main {
    public static void main(String[] args) {
        ModelUmum[] daftarModel = new ModelUmum[3];
        daftarModel[0] = new ModelRegresi("Regresi Linier", 500, 4);
        daftarModel[1] = new ModelKlasifikasi("Decision Tree", 800, 3);
        daftarModel[2] = new ModelDeepLearning("Neural Network", 1000, 5, 3);

        int totalBiaya = 0;
        for (ModelUmum m : daftarModel) {
            m.tampilInfo();
            int biaya = m.hitungBiaya();
            System.out.println("Biaya komputasi: " + biaya);
            System.out.println("---");
            totalBiaya += biaya;
        }
        System.out.println("Total biaya: " + totalBiaya);

        // Neural Network (indeks 2): versi epoch milik kelas induk
        System.out.println("Biaya 10 epoch (Neural Network): " +
daftarModel[2].hitungBiaya(10));

        // Regresi Linier (indeks 0)
        System.out.println("Biaya 10 epoch (Regresi Linier): " +
daftarModel[0].hitungBiaya(10));

        // Versi epoch + batch hanya ada di ModelRegresi, perlu casting
        ModelRegresi regresi = (ModelRegresi) daftarModel[0];
        System.out.println("Biaya 10 epoch batch 50 (Regresi Linier): " +
regresi.hitungBiaya(10, 50));
    }
```

```
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ modul_06 ---
Model: Regresi Linier | Data latih: 500
Jumlah fitur: 4
Biaya komputasi: 1040
---
Model: Decision Tree | Data latih: 800
Jumlah kelas: 3
Biaya komputasi: 4800
---
Model: Neural Network | Data latih: 1000
Jumlah kelas: 5
Jumlah layer: 3
Biaya komputasi: 10600
---
Total biaya: 16440
Biaya 10 epoch (Neural Network): 10000
Biaya 10 epoch (Regresi Linier): 5000
Biaya 10 epoch batch 50 (Regresi Linier): 100
------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------
Total time:  1.941 s
Finished at: 2026-10-23T16:15:38+07:00
------------------------------------------------------------
```

Penjelasan :

1. Kelas induk (ModelUmum)
   Class ini menyimpan atribut yang dimiliki semua jenis model. Keduanya private, sehingga tidak bisa diakses langsung dari *class* lain.
   - Constructor *ModelUmum(String nama, int jumlahDataLatih)* mengisi kedua atribut saat objek dibuat.
   - *tampilInfo()* mencetak nama model dan jumlah data latih.
   - *hitungBiaya()* dan *hitungBiaya(int epoch)* menggunakan nama yang sama dengan parameter berbeda. Versi pertama menghitung data latih 2 kali, versi kedua data latih kali epoch.

2. Kelas turunan pertama (ModelRegresi)
   Kata kunci extends membuat ModelRegresi mewarisi semua method ModelUmum. Untuk atribut barunya ada *jumlahFitur*.
   - Cosntructor memanggil *super(nama, jumlahDataLatih)* untuk mengisi atribut milik induk, lalu mengisi *jumlahFitur* sendiri.
   - *tampilInfo()* adalah overriding. Method ini memanggil *super.tampilInfo()* untuk mencetak baris dari induk, lalu menambahkan baris jumlah fitur.

- *hitungBiaya()* di-override menjadi *super.hitungBiaya() + (jumlahFitur * 10)*, yaitu biaya induk ditambah biaya fitur.
- hitungBiaya(int epoch, int ukuranBatch) adalah method baru dengan nama sama tetapi dua parameter, sehingga menjadi overloading versi ketiga. Yang menjadi catatan adalah pembagian bilangan bulat yang hasilnya nanti dibulatkan ke bawah.

Objek ModelRegresi memiliki tiga versi hitungBiaya : tanpa parameter (override), satu parameter (warisan dari induk), dan dua parameter.

3. Kelas turunan kedua (ModelKlasifikasi)
   - Terdapat atribut baru yaitu *jumlahKelas*.
   - *hitungBiaya(int epoch)* tidak di-override, jadi yang digunakan tetap versi daro ModelUmum.

4. Kelas turunan ketiga (ModelDeepLearning)
   Class ini mewarisi ModelKlasifikasi, bukan ModelUmum langsung. Ini disebut *multilevel inheritance*.
   - Atribur baru ada *jumlahLayar*.
   - Constructor memanggil *super(nama, jumlahDataLatih, jumlahKelas*), yaitu constructor ModelKlasifikasi bukan ModelUmum.
   - *tampilInfo()* memanggil *super.tampilInfo()* milik ModelKlasifikasi. Karena ModelKlasifikasi juga memanggil *super.tampilInfo()* milik ModelUmum, ketiga baris informasi tercetak berurutan secara berantai.
   - *hitungBiaya()* menjadi *super.hitungBiaya() + (jumlahLayer * 200)*. Di sini super menunjuk ke versi ModelKlasifikasi, yang sudah mencakup kali *jumlahKelas*.

5. Main.java
   Array beripe class induk dapat menampung objek dari semua class turunannya, inilah yang disebut dengan polimorfisme.
   Pada loop, variabel m bertipe ModelUmum, tetapi Java memanggil method sesuai tipe objek aslinya saat program berjalan. Sehingga *m.tampilinfo()* mencetak informasi yang berbeda untuk tiap model, padahal kodenya sama.

D. Kesimpulan
   Inheritance (*extends*) membuat kode lebih efisien, karena atribut dan method yang sama cukup dituliskan sekali di *class* induk kemudian diwariskan oleh *class* turunan, termasuk melalui pewarisan bertingkat. Keyword *super* digunakan untuk memanggil constructor dan

method kelas diatasnya, sehingga hasil perhitungan induk dapat dipakai ulang dan dikembalikan di *class* turunan.

Overriding mengubah perilaku method warisan, sedangkan overloading membuat satu nama method memiliki beberapa versi dengan parameter berbeda. Pada polimorfisme memungkinkan objek dari kelas berbeda disimpan dalam satu array bertipe kelas induk dan diproses dengan satu perulangan, dengan method yang dijalankan sesuai tipe objek aslinya. Enkapsulasi tetap terjaga karena semua atribut private dan diakses lewat getter.